

BUT SD

SAE Apprentissage pour l'IA

Cours n°1

Apprentissage automatique & Réseaux de neurones

Marc Métivier



IUT de Paris - Rives de Seine
Université Paris Cité

SAE Apprentissage pour l'IA

Une initiation à l'**Apprentissage Automatique**

- Une initiation au *Machine Learning*
- Une initiation au *Deep Learning*

Organisation générale de l'enseignement :

- 12h de Cours/TP :
 - 4 séances de 3h
- Un projet avec 12h de travail en autonomie
- Un QCM pendant la dernière séance de cours

Plan

1. IA & Machine Learning

2. Apprentissage supervisé

- Exemple de la régression linéaire

3. Les réseaux de neurones

- Neurone artificiel et réseaux de neurones
- 80 ans d'histoire
- Deep Learning
- Biais et éthique

4. Apprentissage dans les réseaux de neurones

- Mise en pratique
- Programmation

L'inévitable IA

Aujourd'hui, l'IA se développe partout

- Traitement des images
 - Reconnaissance des formes, des visages, biométrie
 - Génération d'images selon un style
 - Colorisation réaliste de photos en noir-et-blanc
 - Des floutages / Super-résolution d'images
- Traitement de la vidéo
 - Reconnaissance des éléments d'une vidéos
 - Analyse de comportements en temps-réel
 - Lecture sur les lèvres
 - Génération de vidéos
- Robotique : Drones/Véhicules autonomes, robots chirurgiens
- Agents virtuels : chatbots, assistants, PNJ de jeux vidéo
- Texte : traduction en temps-réel, génération de rapports, synthèse de contenus
- Son : Synthèse vocale, Reconnaissance de la parole, Génération de musique

Une forte accélération récente (liée au Machine Learning et au Deep Learning)

Qu'est-ce que l'IA exactement ?

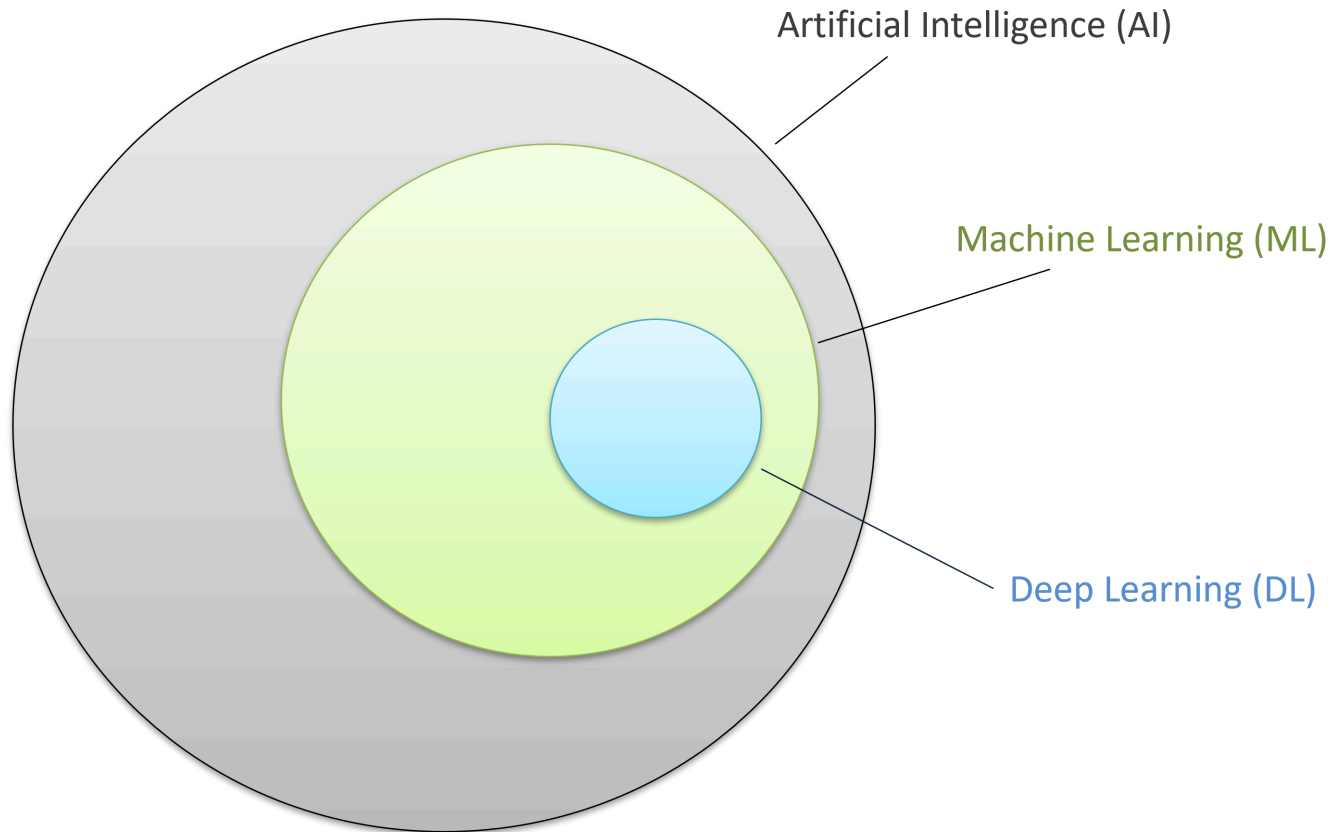
Intelligence Artificielle (IA)

- Un domaine de recherche établi en 1956
 - Il fait partie de l'Informatique
 - Des fondements mathématiques (stats, algèbre linéaire, proba)
 - Des concepts issus des sciences cognitives
- L'étude des méthodes permettant d'effectuer des tâches typiquement associées à l'intelligence :
 - Le raisonnement
 - La résolution de problèmes
 - La perception
 - La prise de décision
 - L'apprentissage (→ Machine Learning)
 - ...
- Le premier intérêt est la compréhension de l'intelligence humaine

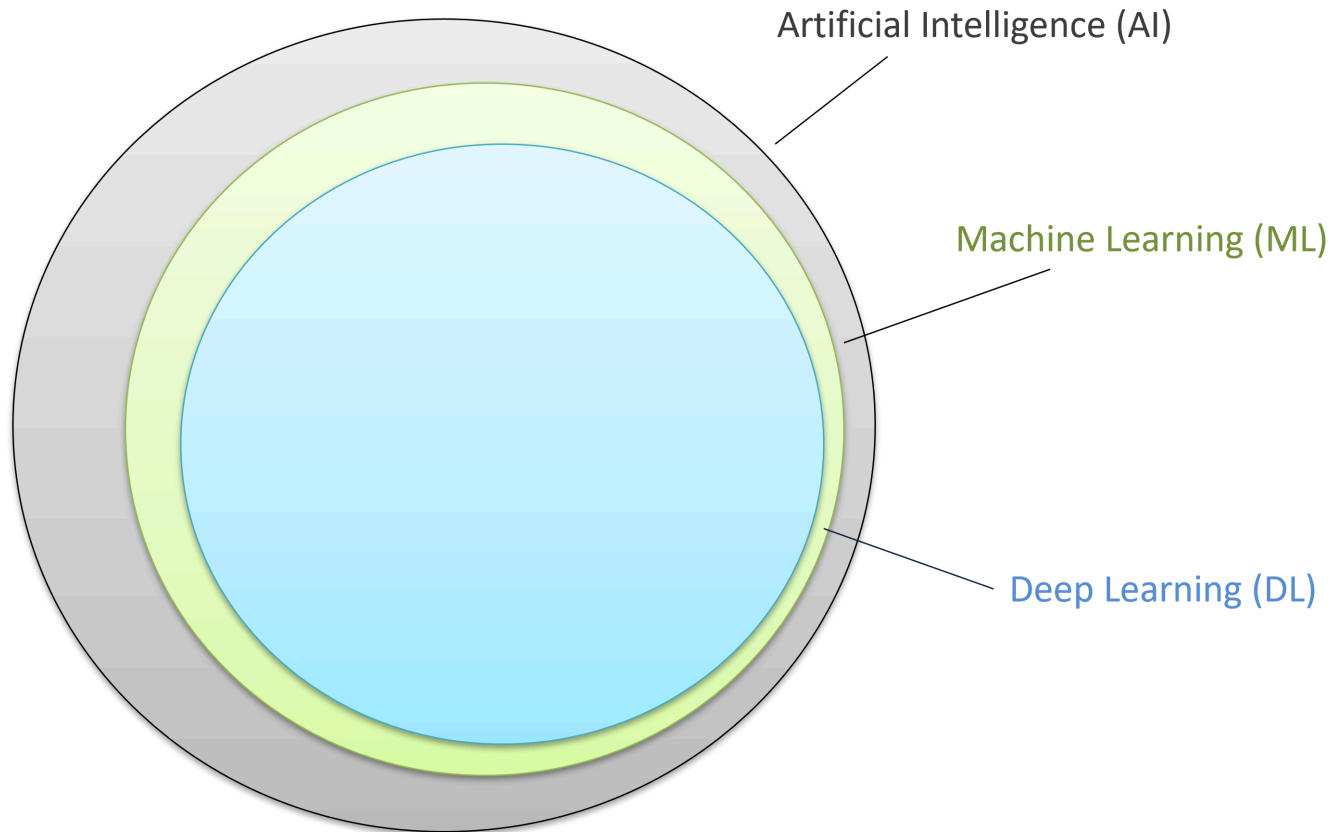


Une fréquente confusion entre IA et Machine Learning / Deep Learning

IA & Machine Learning



Aujourd'hui...



Machine Learning

Machine Learning (ML)

- "**Apprentissage automatique**" en français.
- Un domaine de l'Intelligence Artificielle (IA).
- L'art de construire des systèmes capables d'apprendre à partir des données
- Le principe : le système met à jour ses paramètres pour réduire ses "erreurs"

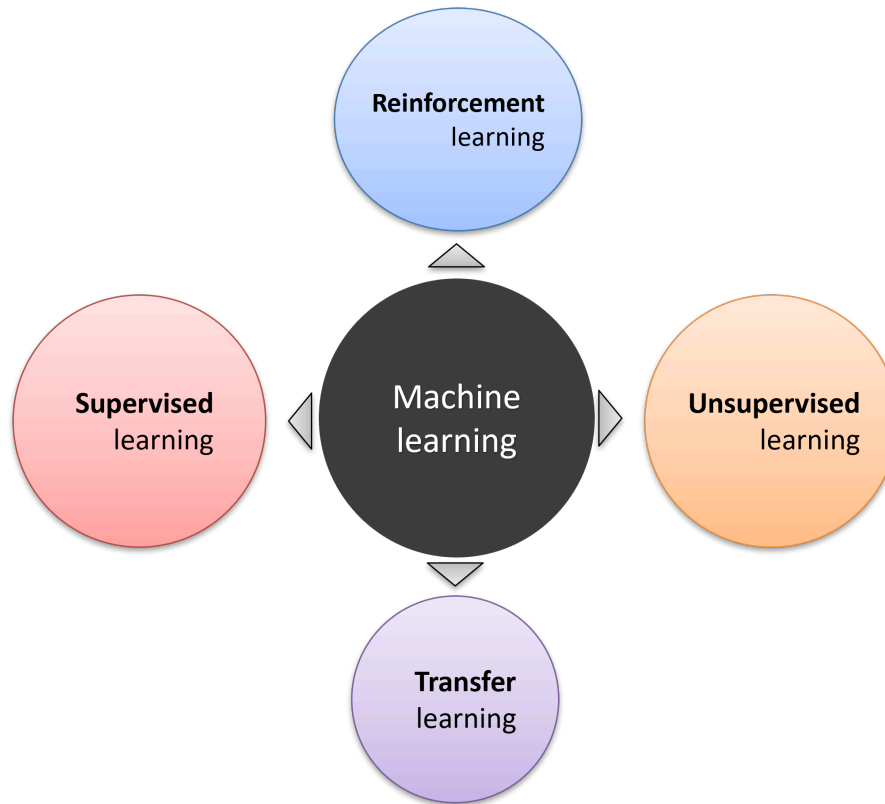
Un domaine très vaste

- De nombreuses méthodes pour de nombreux types de problèmes

Deep learning (DL)

- Des méthodes d'apprentissage utilisant des **réseaux de neurones profonds**
- Un réseau est **profond** s'il possède **plusieurs "couches" de neurones**
- Ce n'est pas un type d'apprentissage mais un **ensemble de méthodes**

Types d'apprentissage



Types d'apprentissage

Supervised Learning / Apprentissage supervisé

- Apprendre sur des données étiquetées
- Modéliser l'association entre une donnée et son étiquette

Classification

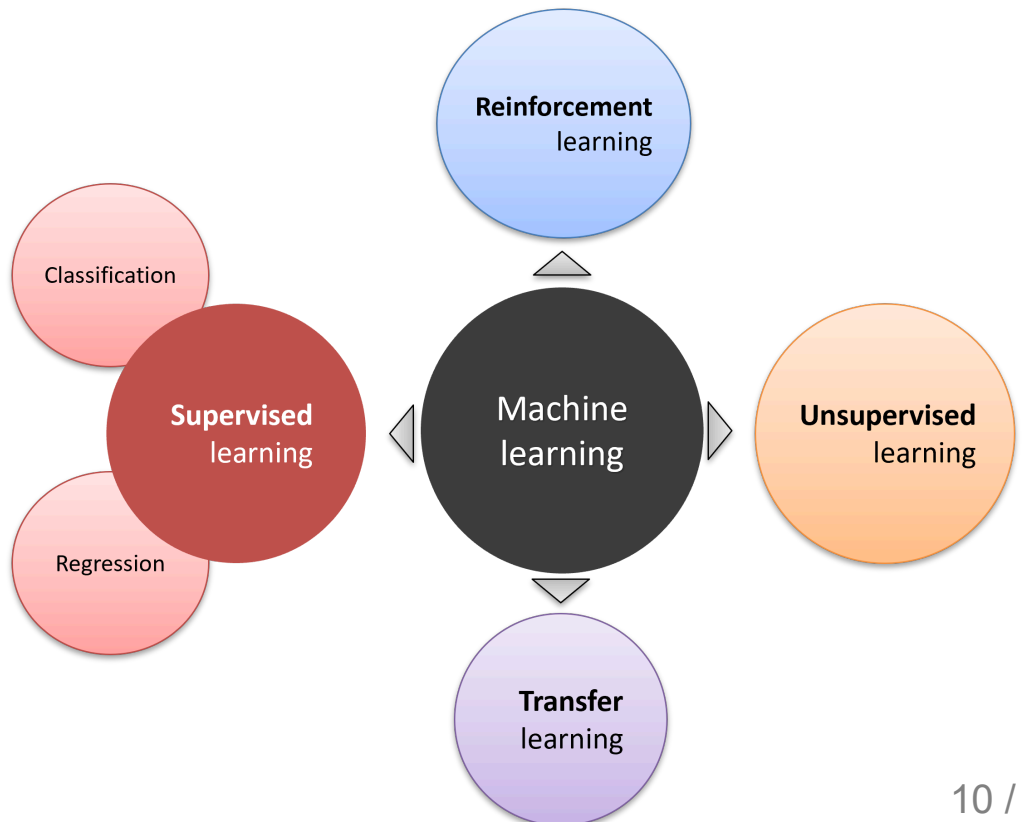
Prédire une information qualitative

Ex : quel animal est sur l'image ?

Régression

Prédire une information quantitative

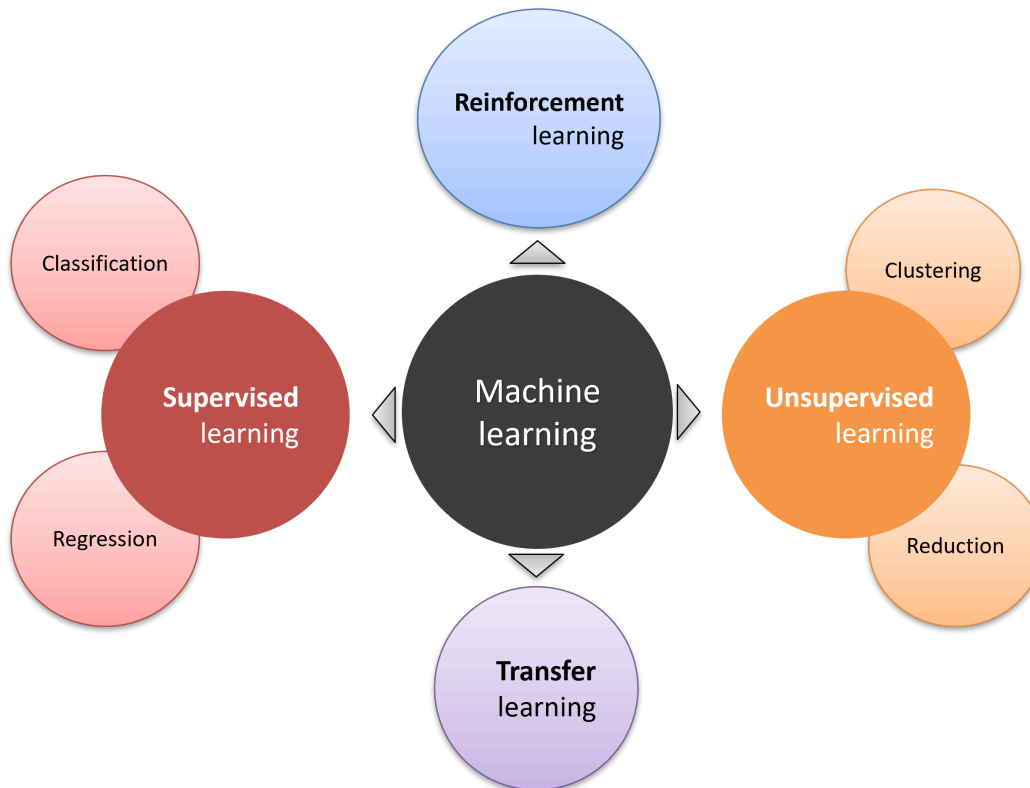
Ex : quel est le prix du logement ?



Types d'apprentissage

Unsupervised Learning / Apprentissage non-supervisé

- Apprendre sur les données seules
- Extraire les liens entre les données



Clustering

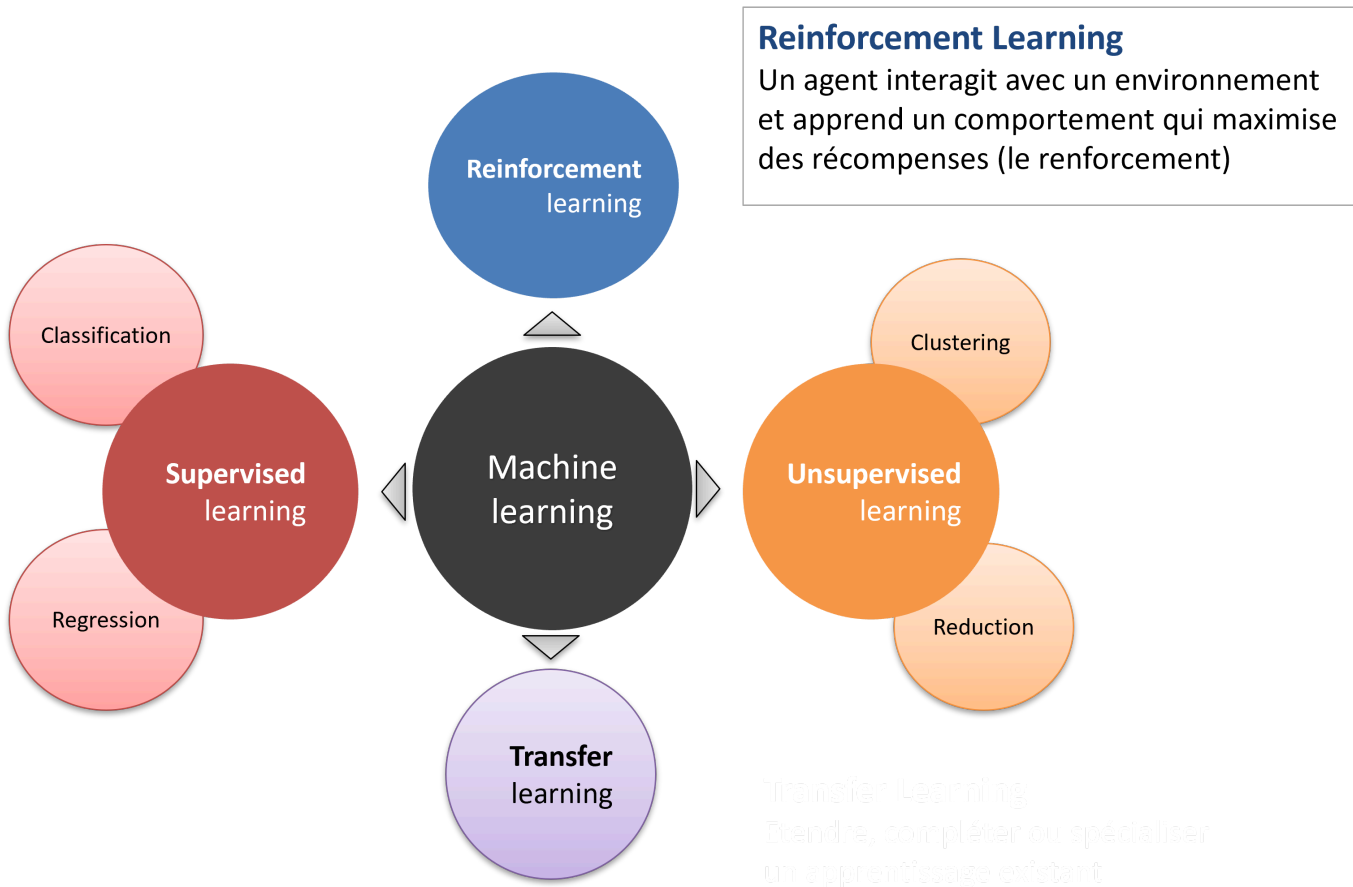
Trouver des relations dans les données
Trouver des classes dans les données

Reduction

Réduire le nombre de dimensions
Compresser sans perdre d'information

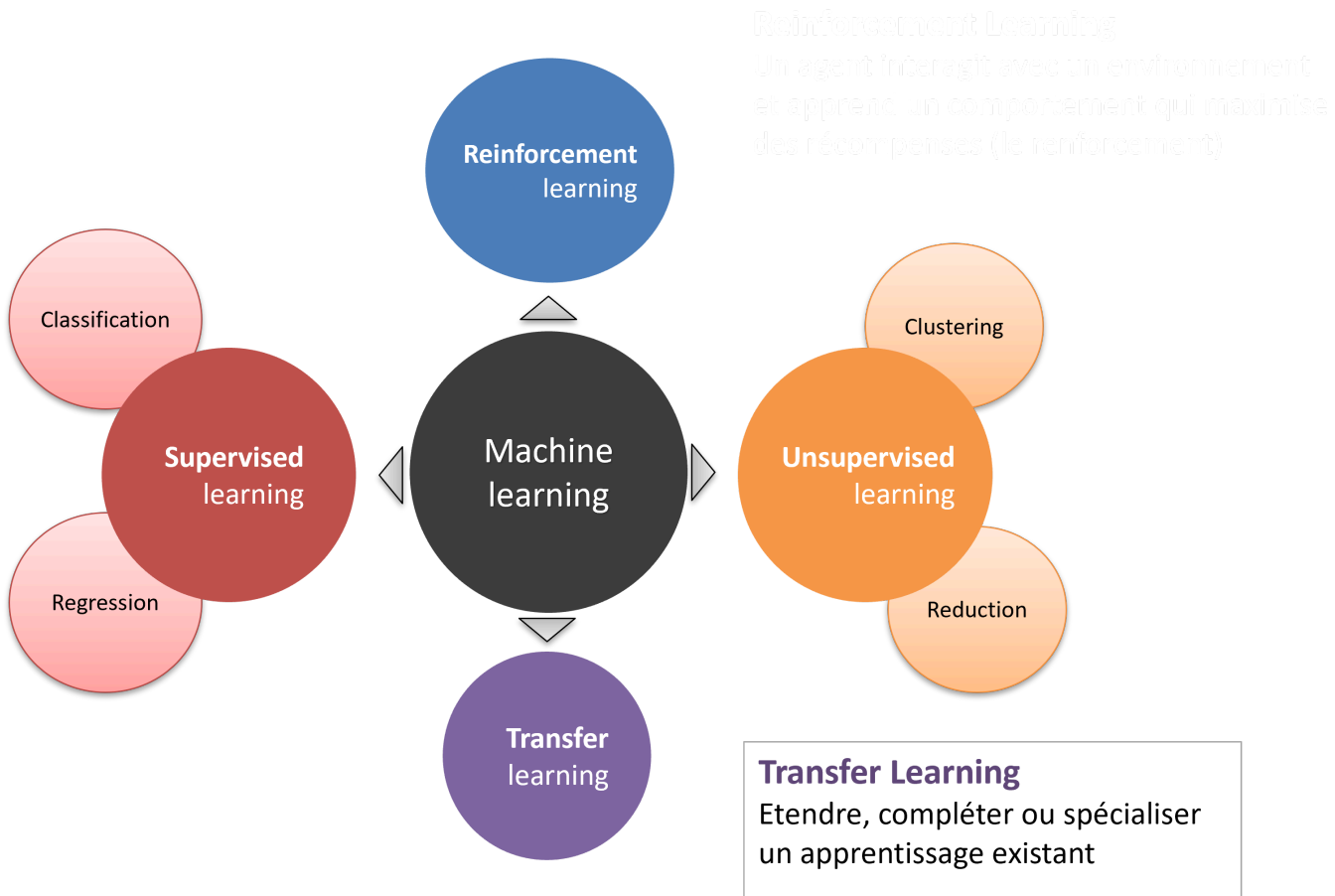
Types d'apprentissage

Reinforcement Learning / Apprentissage par renforcement



Types d'apprentissage

Transfer Learning / Apprentissage par transfert



Apprentissage supervisé

Apprentissage supervisé

Description du problème

- On souhaite étudier un phénomène qui associe des données d'un ensemble $\mathcal{X}$ à des données d'un ensemble $\mathcal{Y}$.
- On dispose de N observations associant une donnée de chaque ensemble :

$$\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(N)}, y^{(N)})\}$$

- On veut comprendre et modéliser ce phénomène :
 - Savoir prédire la donnée de $\mathcal{Y}$ à partir de celle de $\mathcal{X}$
 - Si j'observe une nouvelle donnée dans $\mathcal{X}$ quelle est sa valeur de $\mathcal{Y}$ associée ?



Le but est de construire un modèle expliquant les données.

Remarques

C'est de l'**apprentissage supervisé** car $\mathcal{Y}$ existe.

Le type d'apprentissage supervisé dépend de $\mathcal{Y}$:

- Si $\mathcal{Y} \subset \mathbb{R}$: problème de **régression**
- Si $\mathcal{Y} = \{1, \dots, m\}$: problème de **classification**

$\mathcal{X}$ peut être un espace de dimension n

- Chaque donnée d'entrée est définie par n éléments
- Chaque donnée $x \in \mathcal{X}$ est un vecteur de n valeurs

$$x = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$$

Modélisation

Le **modèle** est une fonction mathématique de $\mathcal{X} \rightarrow \mathcal{Y}$

En général

- On fait l'hypothèse qu'il existe une fonction inconnue f telle que :

$$y = f(x) + \epsilon$$

- ϵ est un "bruit"
 - Il sert à prendre en compte les imperfections dans les observations
- On cherche à estimer la fonction f .
- Il existe de nombreux modèles et méthodes possibles.

Exemple de la régression linéaire

On cherche à modéliser la fonction f par une droite

$$f(x) = a \cdot x + b$$

- Cette droite est notre modèle.
- Les paramètres du modèle forment un vecteur θ avec : $\theta = \begin{pmatrix} a \\ b \end{pmatrix}$

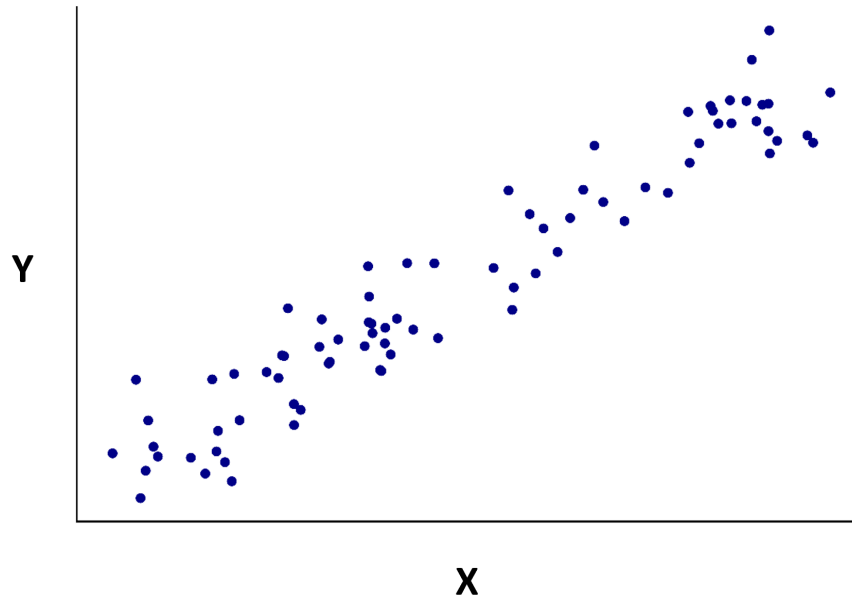
Remarque : cette représentation fonctionne aussi en dimension $n > 1$.

- La droite est alors une "hyper-droite"
- a devient un vecteur :

$$a = \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{pmatrix} \text{ et donc } \theta = \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \\ b \end{pmatrix}$$

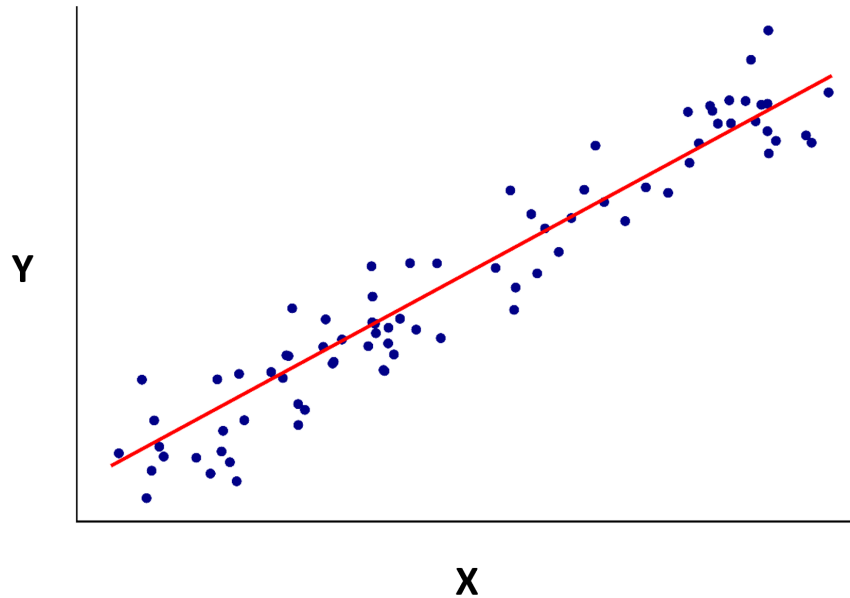
Exemple de la régression linéaire

On étudie un phénomène pour lequel on dispose des observations suivantes



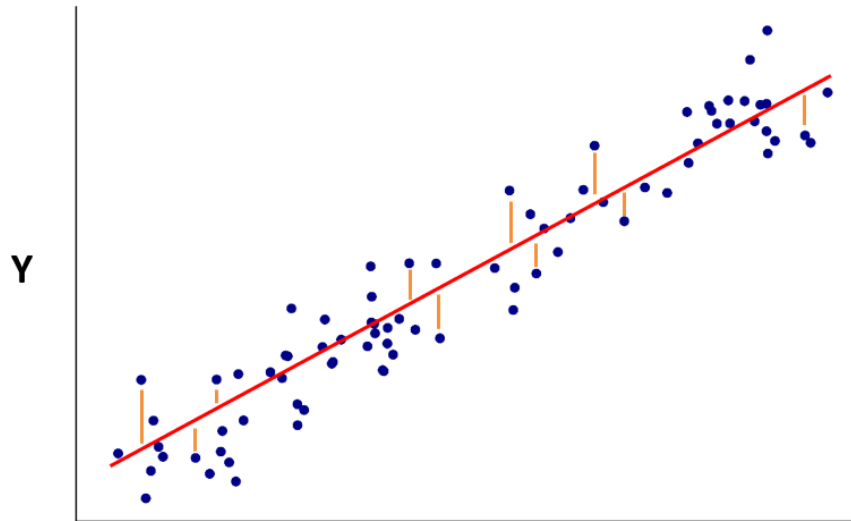
Exemple de la régression linéaire

On recherche une droite passant le "plus près possible" des points



Exemple de la régression linéaire

Le "plus près possible" signifie "minimiser" la distance



III ➡ Pour chaque point (x, y) , on veut minimiser la distance entre $f(x)$ et y .

Bonne nouvelle : il existe une solution directe !

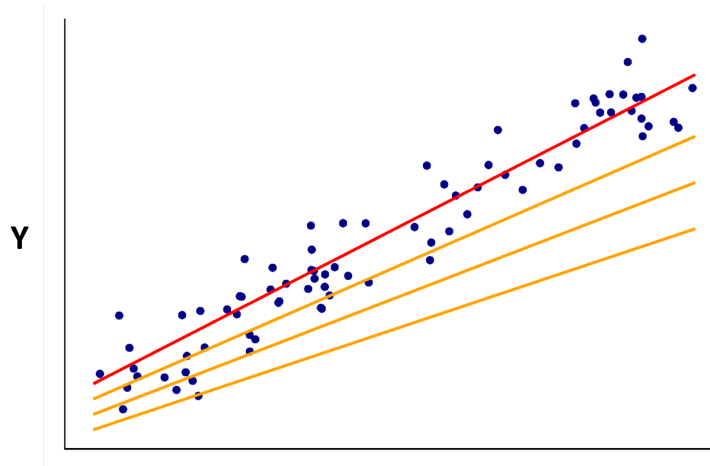
Mauvaise nouvelle : son calcul est de complexité en $O(N^3)$! 

Optimisation

En pratique, on se tourne vers des algorithmes d'optimisation

On recherche θ par un processus itératif :

- On choisit un premier vecteur θ arbitrairement
- On répète :
 - Mesurer la qualité du modèle
 - Modifier les paramètres dans la direction d'une meilleure qualité
- Le but est que la qualité augmente au fur et à mesure des répétitions



La fonction de coût du modèle

Qualité du modèle ?

III ➡ On utilise une **fonction de coût** ("loss" en anglais)

Quel coût pour la régression linéaire ?

III ➡ L'erreur quadratique moyenne :

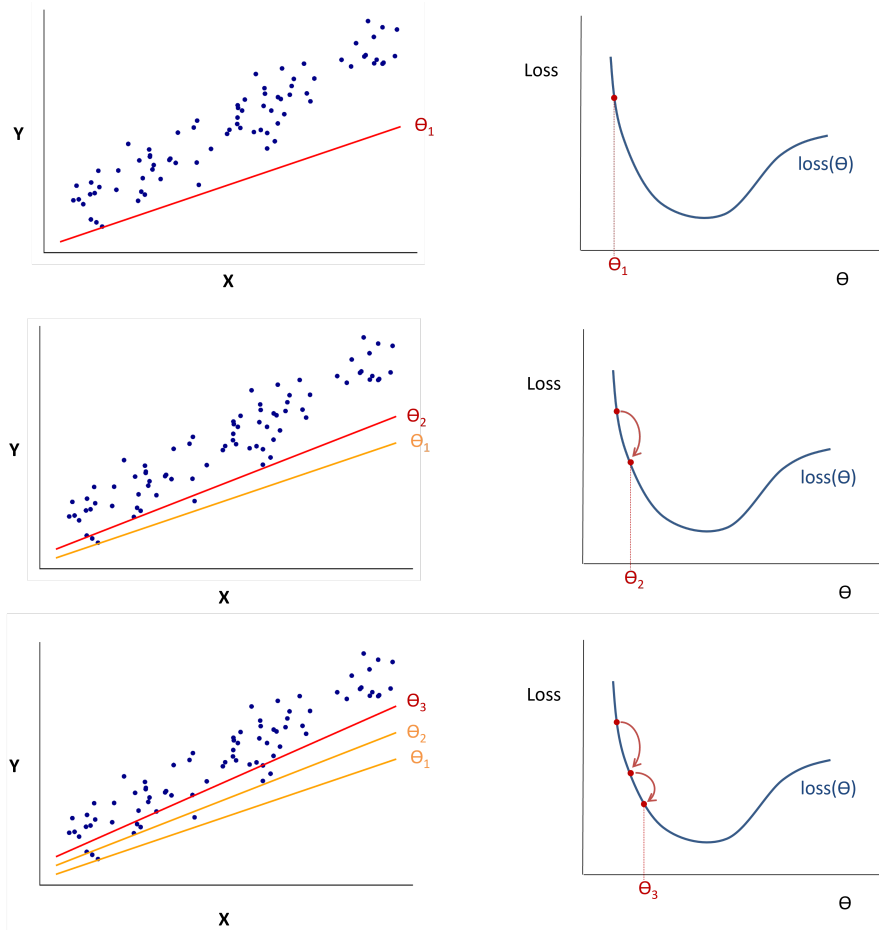
$$\text{loss}(\theta) = \frac{1}{N} \sum_{i=1}^N [f_{\theta}(x^{(i)}) - y^{(i)}]^2$$

En anglais : "Mean Squared Error" (MSE)

L'idée est d'explorer l'espace de paramètres θ pour trouver ceux dont le coût est minimal

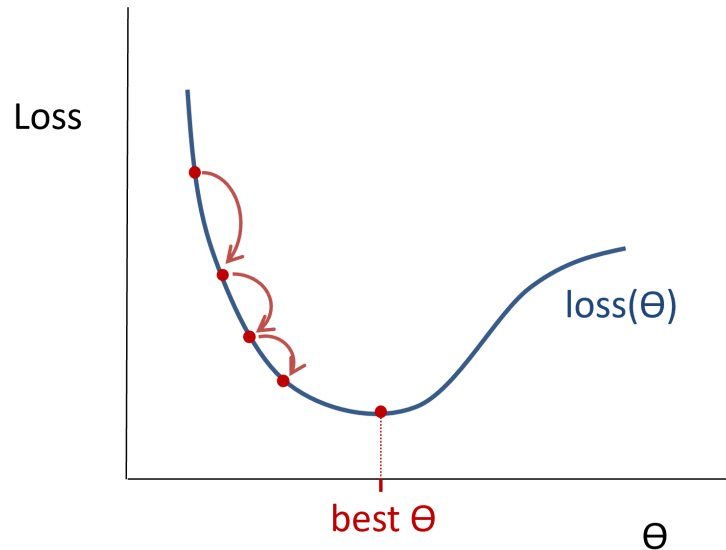
Optimisation

Rechercher les paramètres de coût minimum



Optimisation

Le but est d'évoluer vers le coût minimal à chaque itération

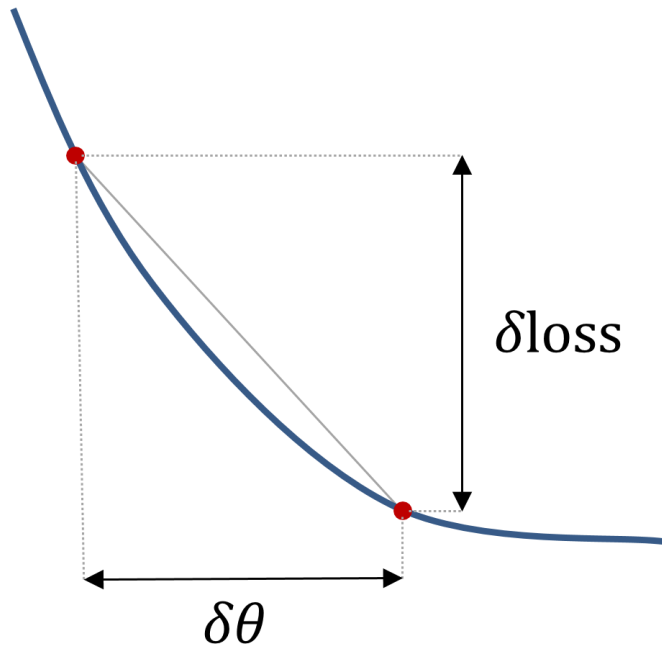


Mais comment modifier les paramètres pour que le coût diminue ?

III ➡ La **descente de gradient**

La descente de gradient

Si on change θ de $\delta\theta$, on modifie $\text{loss}(\theta)$ de δloss



$$\text{gradient} = \frac{\delta\text{loss}}{\delta\theta}$$

- Il donne la **pente** de la fonction au point θ .
- Une direction pour modifier θ afin de diminuer le coût.
- A chaque iteration :

$$\theta \leftarrow \theta - \eta \cdot \frac{\delta\text{loss}}{\delta\theta}$$

où η un **taux d'apprentissage** (*learning rate* en anglais).

Ce processus est appelé **descente de gradient**.

Retour sur la régression linéaire

Descente de gradient sur l'erreur quadratique

$$\text{loss}(\theta) = \frac{1}{N} \sum_{i=1}^N [f_{\theta}(x^{(i)}) - y^{(i)}]^2 = \frac{1}{N} \sum_{i=1}^N [Err^{(i)}]^2$$

Le gradient est le vecteur de dérivés partiels (une pour chaque paramètre)

$$\text{gradient} = \nabla \text{loss}(\theta) = \begin{bmatrix} \frac{\partial}{\partial \theta_0} \text{loss}(\theta) \\ \frac{\partial}{\partial \theta_1} \text{loss}(\theta) \\ \vdots \\ \frac{\partial}{\partial \theta_n} \text{loss}(\theta) \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial b} \text{loss}(\theta) \\ \frac{\partial}{\partial a_1} \text{loss}(\theta) \\ \vdots \\ \frac{\partial}{\partial a_n} \text{loss}(\theta) \end{bmatrix} = \begin{bmatrix} \frac{2}{N} \sum_{i=1}^N Err^{(i)} \\ \frac{2}{N} \sum_{i=1}^N x_1^{(i)} \cdot Err^{(i)} \\ \vdots \\ \frac{2}{N} \sum_{i=1}^N x_n^{(i)} \cdot Err^{(i)} \end{bmatrix}$$

$\text{gradient} = \frac{2}{N} X^T \cdot (X \cdot \theta - Y)$ avec X et Y les vecteurs des x et des y .

Modèles des données

Les méthodes d'apprentissage produisent des modèles.

- La régression linéaire n'est qu'un exemple de régression.

Il existe d'autres modèles de régression.

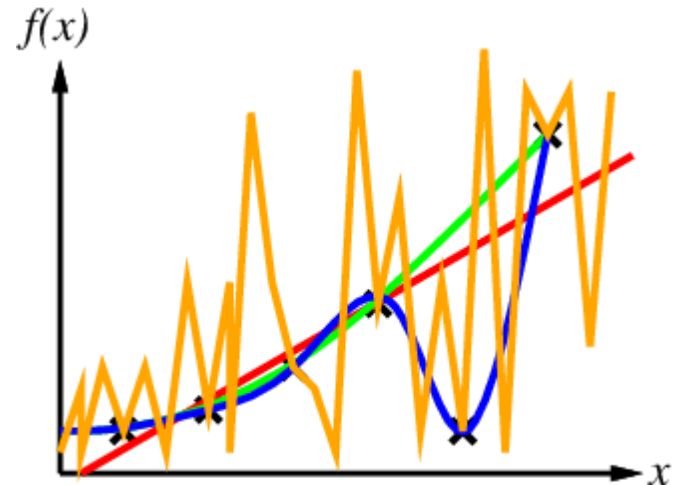
- La régression polynomiale modélise les données par un polynôme d'ordre k :

$$P_k(x) = a_0 + a_1 \cdot x + a_2 \cdot x^2 + \dots + a_k \cdot x^k = \sum_{i=1}^k a_i \cdot x^i$$

Mais qu'est-ce qu'un bon modèle ?



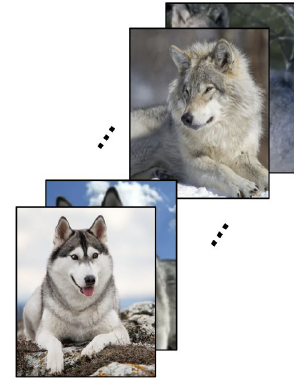
Sur-apprentissage (over-fitting) !



Attention aux biais

Les données peuvent introduire des biais significatifs

- « Huskies vs Wolves » [Ribeiro et al. 2016]
 - Le système a appris qu'une photo représentait un loup **s'il y avait de la neige**



Plus grave...

Microsoft débranche son robot intelligent devenu raciste et pro-nazis en 24 heures

25 mars 2016, 10:58



Aux USA, un logiciel aide les juges à devenir racistes



Guillaume Champeau · 24 mai 2016 · Société

ALGORITHME DE RECRUTEMENT

Quand le logiciel de recrutement d'Amazon discrimine les femmes

En 2014, le géant du e-commerce a voulu confier ses candidatures à un algorithme, mais celui-ci a commencé à écarter les profils féminins.

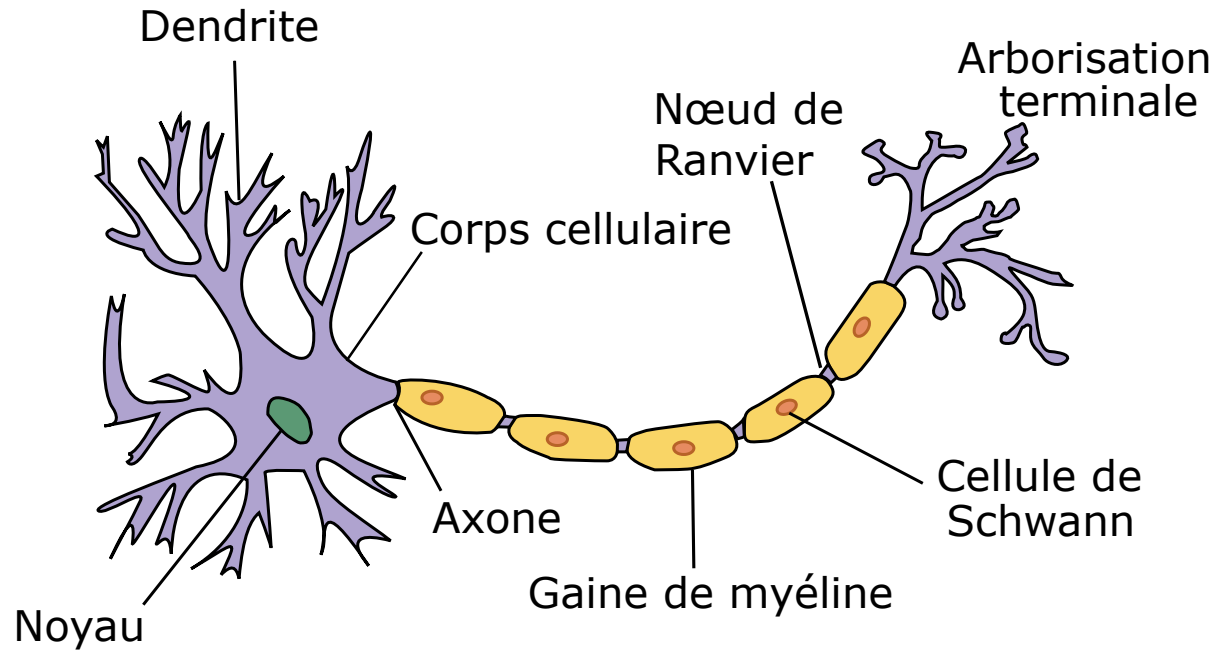
Source : *Les Echos*

Amazon a dû désactiver une IA qui discriminait les candidatures de femmes à l'embauche

Source : *Numerama*

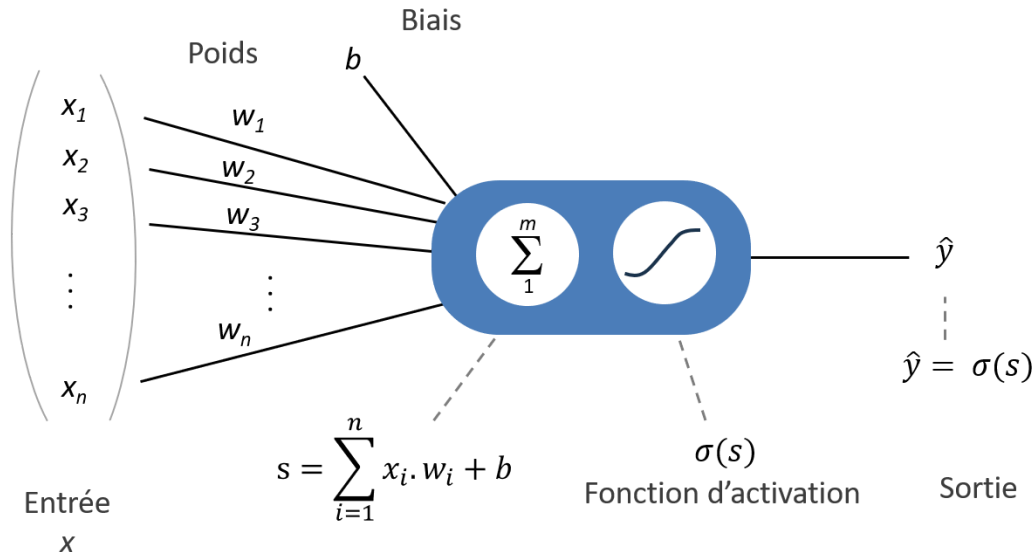
Réseaux de neurones

Neurone biologique



Weft, CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=1527770>

Le neurone formel



$$\Rightarrow \hat{y} = \sigma\left(\sum_{i=1}^n x_i \cdot w_i + b\right)$$

Remarques sur les notations :

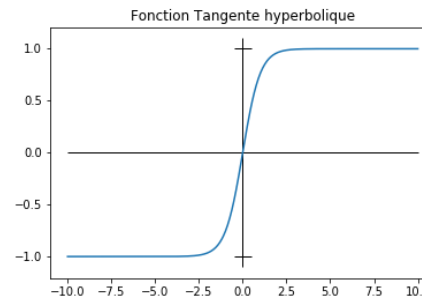
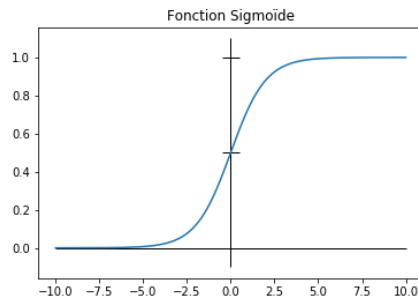
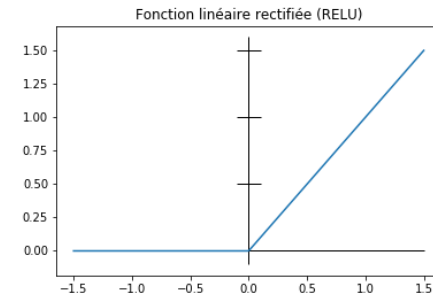
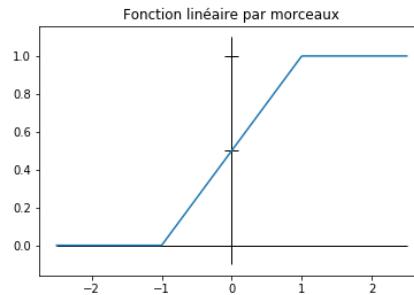
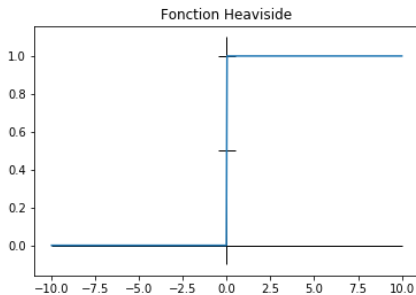
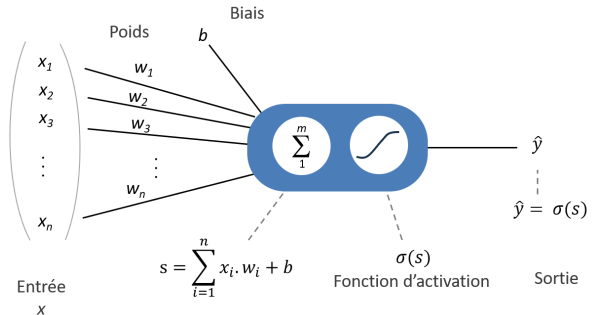
- Si x et w sont les vecteurs des (x_i) et (w_i) $\rightarrow \hat{y} = \sigma(x \cdot w + b)$
- On peut ajouter b dans w ($w_0 = b$ et $x_0 = 1$) $\rightarrow \hat{y} = \sigma(x \cdot w)$

Fonctions d'activation

La fonction $\sigma : \mathbb{R} \rightarrow \mathbb{R}$

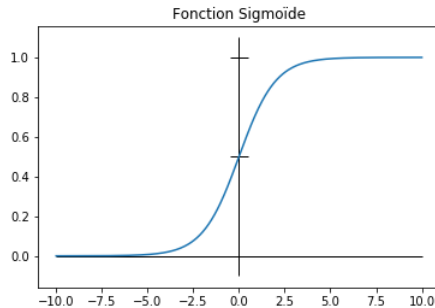
- Elle définit l'aspect des sorties

Quelques fonctions classiques :



Lien avec la régression logistique

Prenons la fonction sigmoïde



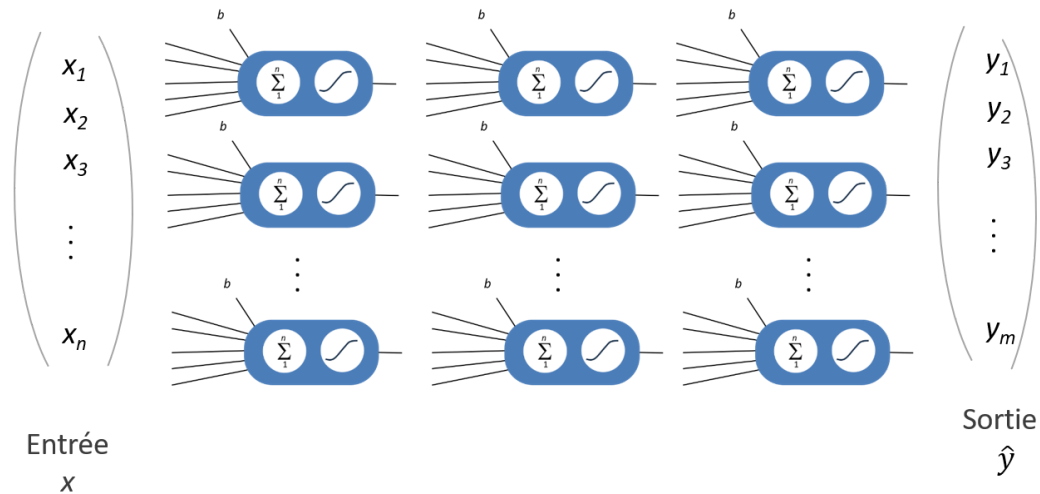
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Cela permet de classer les données en 2 classes (0 ou 1)
- La sortie s'interprète comme la probabilité d'appartenir à la classe 1

III ➡ On obtient exactement la **Régression logistique** !

Réseau de neurones

Un ensemble de neurones formels connectant les entrées aux sorties



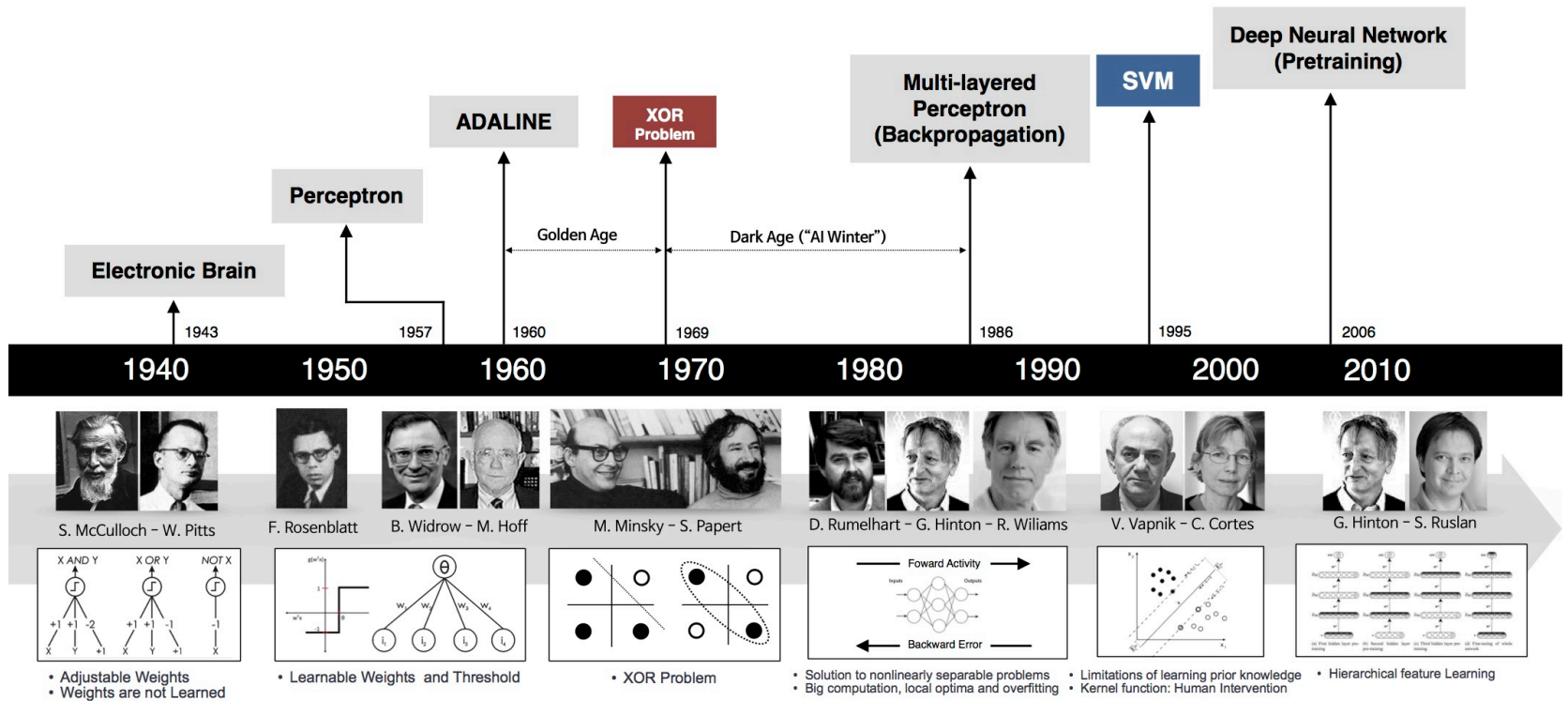
Les entrées de chaque neurone sont connectées :

- soit aux données d'entrées
- soit à la sortie d'un autre neurone

De nombreuses architecture possibles

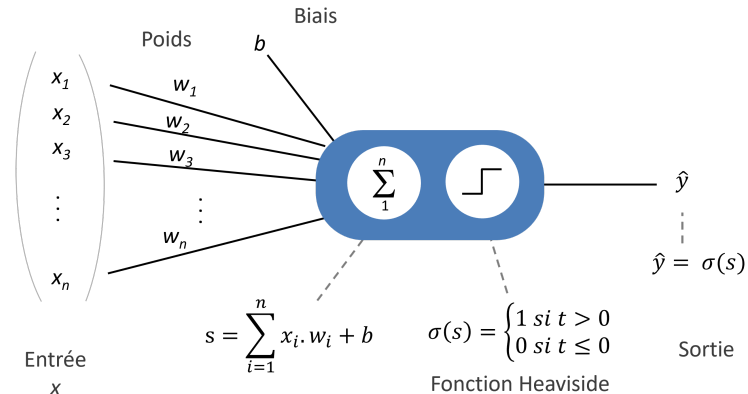
Un peu d'histoire

80 ans d'histoire



http://beamlab.org/deeplearning/2017/02/23/deep_learning_101_part1.html

Le Perceptron



Inventé par Frank Rosenblatt en 1957

- C'est un séparateur linéaire dans l'espace des entrées
- Utilisé avec succès sur le jeu de données IRIS (datant de 1936)
 - Reconnaître une Iris en fonction de la largeur et la longueur des pétales

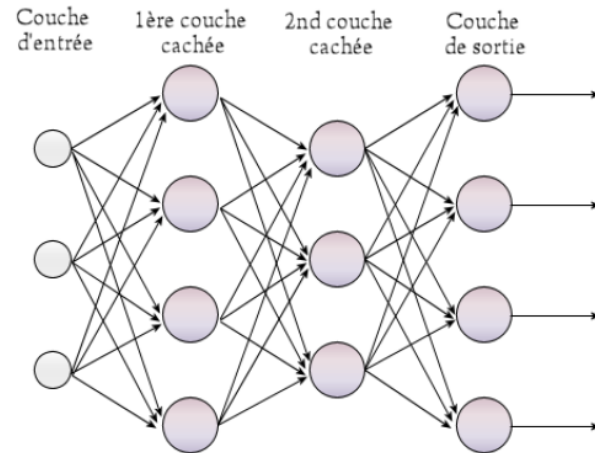
En 1969, Marvin Minsky montre les limites du Perceptron

- L'opérateur binaire XOR ne peut pas être appris par un Perceptron
- Début d'une période appelée "l'hiver de l'IA".

Multi-Layered Perceptron (MLP)

Perceptron multi-couches

- *Feedforward Neural Network*
- Des couches de neurones interconnectées
- Les couches de neurones internes sont les "couches cachées"



HRcommons, Public domain, via Wikimedia Commons

Une architecture devenue fonctionnelle grâce à l'algo **Backpropagation**

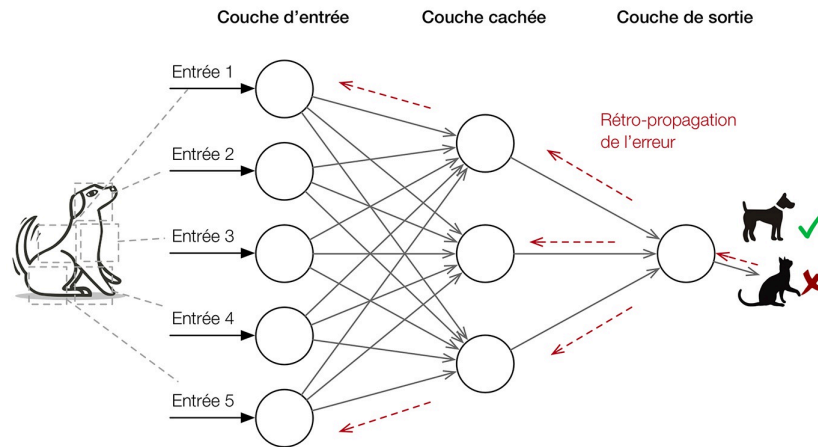
- Rétro-propagation du gradient de l'erreur (*Backprop*)

Un approximateur universelle

- Toute fonction peut être approximée par un MLP avec une couche cachée (de taille "suffisante")

Le principe du *Backprop*

La rétro-propagation du gradient de l'erreur



D Cardon, JP Cointet, A Mazieres (2018), La revanche des neurones

Des allers et retours

- L'algorithme propage des calculs dans un sens et dans l'autre, à répétition
 - **Forward pass** : calcul des sorties à partir des entrées
 - **Backward pass** : mise à jour des paramètres des sorties jusqu'au entrées
- Mise à jour des paramètres : retro-propagation du gradient de l'erreur observée

Deep Learning (DL)

On parle de *Deep Neural Network* (DNN) s'il y a *plusieurs couches cachées*

Le *Deep Learning* explose réellement après 2012

- Avant les SVM sont prédominants dans le ML.
- 2012 : AlexNet (Krizhevsky, Sutskever, Hinton)
 - Un DNN qui obtient 15% d'erreur à ILSVRC 2012
 - Tous les autres sont non DNN et font plus de 26% (4% pour les humains)

Les DNN sont sans garantie mathématique

- Mais ils se montrent bien meilleurs que les autres

Quelques architectures

- 2012 : AlexNet
- 2013 : VAEs
- 2014 : GANs
- 2015 : ResNet
- 2016 : AlphaGo
- 2017 : Transformers
- 2018 : LLMs (Bert, GPT-1)
- 2019 : StyleGANs
- 2021 : AlphaFold2, DALL-E
- 2022 : ChatGPT, Stable Diffusion, MidJourney
- 2023 : Claude, Copilot, Gemini, Grok, Llama, Mistral
- 2024 : Veo, Sora
- 2025 : DeepSeek

Progression

La taille des datasets disponibles et la puissance des calculateurs

- La taille des datasets a été multiplié par 1M en 25 ans
- La puissance des calculateurs a été multipliée par 1M Flops en 25 ans

Le nb de paramètres explose (estimations)

- 2019 : GPT-2 a 1.5G paramètres
- 2020 : GPT-3 a 175G paramètres
- 2023 : GPT-4 a 1000G paramètres
- 2025 : GPT-5 a 1.8T paramètres

Un intérêt public sans précédent

- Il a fallu 5 jours à ChatGPT (2022) pour atteindre 1M d'utilisateurs
- En comparaison :
 - 2.5 mois pour Instagram
 - 5 mois pour Spotify
 - 10 mois pour Facebook ...

De nombreux problèmes éthiques

- Reproduction et augmentation de biais sociaux (Discriminations, etc.)
- Sécurité des données
 - Apprentissage sur des données privées ou protégées
 - Garantie de la propriété intellectuelle
- Transparence & Explicabilité : l'effet boîte noire
- Impact sur les emplois et les métiers
 - Remplacement de compétences par l'IA
 - Emplois devenant obsolètes
 - Perte de l'expertise humaine à long-terme
- Décisions morales & Responsabilités
 - Accidents avec des véhicules autonomes (<http://moralmachine.mit.edu>)
- Deep Fakes & Manipulation de l'information

Les principes éthiques de l'IA

Les 9 principes éthiques de l'IA :

- Respect des droits de l'Homme
- Protection des valeurs humaines
- Responsabilité professionnelle
- Contrôle humain
- Équité et non-discrimination
- Transparence et Explicabilité
- Sûreté et sécurité
- Reddition de comptes
- Vie privée

Un cadre juridique en pleine évolution

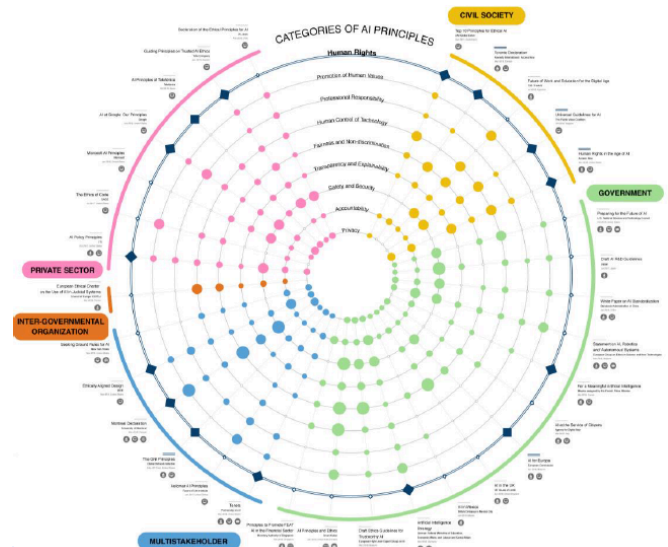
- Interdiction des décisions automatisées
- Obligations : transparence, données non biaisées, exactitude, cybersécurité

Il est urgent de légiférer car l'IA est poussée en avant par des intérêts privés

PRINCIPLED ARTIFICIAL INTELLIGENCE

A Map of Ethical and Rights-Based Approaches July 4, 2019

Authors: Jessica Fjeld, Hannah Hilligoss, Nele Achten, Maia Levy Daniel, Joshua Feldman, Sally Kagay
Design: Arushi Singh (arushisingh.net)



Apprentissage dans les réseaux de neurones

Apprentissage dans les RN

La rétro-propagation du gradient de l'erreur

- On définit une **fonction de coût** (loss) en fonction du type de sorties
- **Descente de gradient** sur chaque couche du réseau en partant de la fin

III ➡ Une correction de l'erreur observée est retro-propagée dans les paramètres

Algorithme Backprop

Une **itération** (**epoch** en anglais) :

- **Obtenir les sorties** $\hat{y}$ associées aux données x (*forward pass*)
- **Calcul du coût** $\mathcal{L}(\theta)$: par comparaison des $\hat{y}$ et des y
- **Calcul du gradient** : $\frac{\delta \mathcal{L}(\theta)}{\delta \theta}$
- **Mises à jour** par rétro-prop. (*backward pass*) : $\theta \leftarrow \theta - \eta \cdot \frac{\delta \mathcal{L}(\theta)}{\delta \theta}$

Répéter les **itérations** jusqu'à atteindre une **condition de fin**

Architecture des sorties et coûts

Cas d'un MLP avec des entrées de taille n et des sorties de taille m

Coût global : la moyenne des coûts unitaires

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_i \ell(y^{(i)}, f(x^{(i)}))$$

RN pour la régression

- Couche de sortie : **pas de fonction d'activation**
- Coût unitaire habituel : **erreur quadratique** (mean squared error, MSE)

$$\ell(y, \hat{y}) = \|y - \hat{y}\|_2^2 = \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

RN pour la classification

- Couche de sortie : on applique la fonction **softmax**
- Coût unitaire habituel : **entropie croisée** (cross entropy)

$$\ell(y, \hat{y}) = - \sum_{i=1}^m y_i \cdot \log \hat{y}_i$$

La fonction Softmax

Avoir une distribution de probabilités en sortie

- Vecteur de sortie du réseau : $\hat{y} = (\hat{y}_1, \dots, \hat{y}_m)$
- On voudrait convertir ces valeurs en probabilités $(\hat{p}_1, \dots, \hat{p}_m)$

$$\hat{p}_i \in [0, 1], \forall i \in \{1, \dots, m\} \quad \text{et} \quad \sum_{i=1}^m \hat{p}_i = 1$$

Application de Softmax :

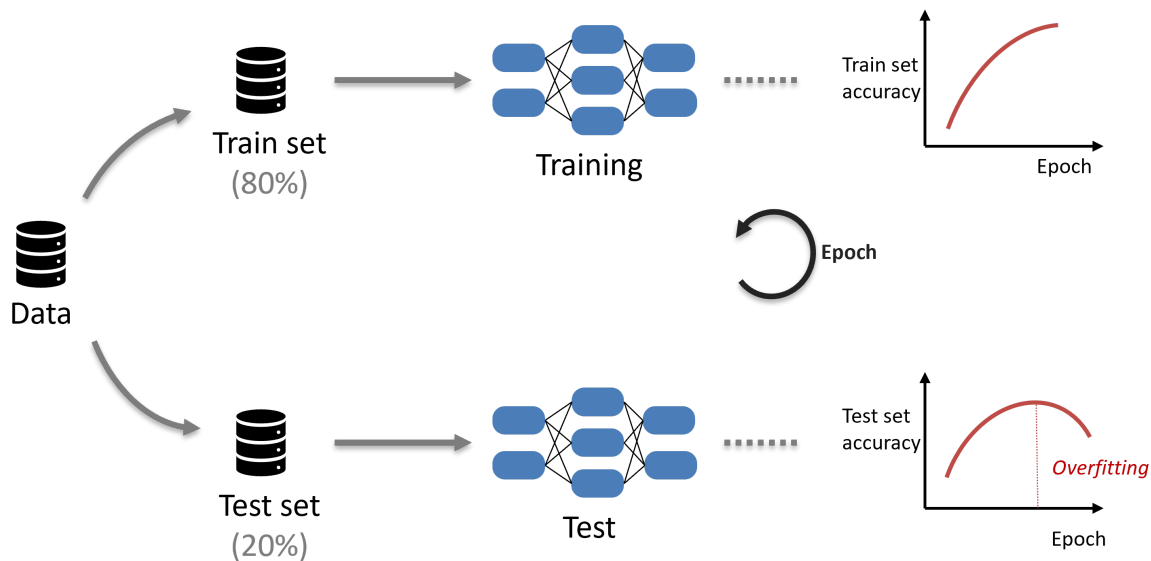
$$\hat{p}_i = \frac{e^{\hat{y}_i}}{\sum_{j=1}^m e^{\hat{y}_j}}, \forall i \in \{1, \dots, m\}$$

- Ce n'est pas une fonction d'activation (car somme sur toutes les sorties)
- C'est une sorte de nouvelle couche placée est sortie du le réseau

Mise en pratique de l'apprentissage

Quand arrêter l'apprentissage ? Comment l'évaluer ?

En pratique, on sépare l'ensemble de données en deux ensembles :



- La séparation se fait aléatoirement
- Le réseau n'est mis à jour qu'avec l'ensemble d'apprentissage, pas celui de test
- La précision (*accuracy*) est le taux de bonne valeur prédit par le réseau

La descente de gradient

Batch Gradient Descent (Batch GD)

- Une seule rétro-propagation du gradient par itération (epoch)
- Le gradient est calculé sur le coût global sur toutes les données

Stochastic Gradient Descente (SGD)

- Si on a trop de données pour faire une passe sur toutes les données
- Calcul et rétro-propagation du gradient à chaque donnée
- On utilise uniquement le coût unitaire

Mini-batch Gradient Descente : méthode intermédiaire

- Comme le Batch GD mais sur des sous-ensembles des données
- Chaque sous-ensemble est appelé "un batch"
- Une rétro-propagation du gradient pour chaque batch

Hyper-paramètres

Paramètres

- Les éléments du modèle mis à jour par l'apprentissage
- Poids et biais des neurones

Hyper-paramètres

- Ce sont les paramètres de la méthode d'apprentissage
- Ils sont à définir avant d'exécuter l'apprentissage
- Architecture :
 - le nombre de couches de neurones
 - le nombre de neurones dans chaque couche
 - les fonctions d'activations
- Apprentissage :
 - le taux d'apprentissage η
 - la taille des batches
 - le nombre d'itérations d'apprentissage (epochs)

Les étapes classiques

1. Imports et initialisations
2. Chargement des données (dataset)
3. Préparation des données
 - Ensemble d'apprentissage (X_{train} , Y_{train})
 - Ensemble de test (X_{test} , Y_{test})
4. Création du modèle
5. Entraînement du modèle
6. Evaluation du modèle

Un monde Python centré

- La base : Python, Numpy, Jupyter
- Matériel : des GPU, des GPU et des GPU...
- Bibliothèques : deux mondes

Keras et TensorFlow

- Keras
 - API de haut niveau
 - MIT Licence
 - Dans TF depuis 2017
- TensorFlow (TF)
 - Framework le plus utilisé
 - Soutenue par Google
 - API de bas niveau
 - Apache Licence

PyTorch avec Lightning

- PyTorch Lightning
 - Interface pour PyTorch
 - API de haut niveau
 - Stable et propre
- PyTorch
 - Issue de la librairie Torch
 - Soutenue par Facebook
 - BSD Licence

